

ESPECIFICACIÓN

Auditoría del código fuente — dónde está el valor, dónde el riesgo y un agujero en la verificación

Fecha: 18-09-2026 · Commit: `d16be0b` · Alcance: `pipeline_comercial.jsx`, el PRODUCTO Marco: skill `agentic-repo-bootstrap-v2` · Verificación previa: los seis pasos en verde (151/151 gates, 140/140 la suite, 29/29 e2e)

Los dos informes anteriores midieron el REPOSITORIO y dejaron el código afuera a propósito. Este mide el código. Todo lo que sigue sale de contar, no de opinar; los comandos están en §8.

Lo que este documento NO dice. No dice que haya que refactorizar. Un fuente de 26.233 líneas que pasa 140 casos de dominio, 151 gates y 29 casos de pantalla no tiene un problema de calidad: tiene una FORMA, y esa forma tiene consecuencias medibles sobre lo que cuesta cambiarlo. De eso trata lo que sigue.

1. La forma, en seis números

MEDIDA	VALOR	QUÉ SIGNIFICA
Líneas	26.233	un solo archivo, sin bundler
Funciones y componentes	1.175 (633 a columna 0, 542 anidadas)	
Tamaño mediano de una función	6 líneas (p90: 44)	la unidad de trabajo típica es chica
Funciones de una sola línea	463	39 % del total
El 1 % más grande (11 funciones)	25 % del fuente	toda la masa está en once sitios
Bloques duplicados (ventana de 15 líneas)	0 · con ventana de 8: 0,4 %	

Las dos primeras filas describen un archivo enorme. Las cuatro siguientes describen un archivo **bien factorizado**: la mediana de seis líneas y la duplicación casi nula no son lo que uno encuentra en un fuente que creció sin control. Las dos cosas son ciertas a la vez, y la tensión entre ellas es el tema de §3.

Composición: 19 % comentario, 26 % líneas con JSX, 12 % con `style` inline, 1 % vacías. Un quinto del archivo explica por qué el código es como es — y esos comentarios son la memoria que hizo posible partir `CLAUDE.md` sin perder nada.

2. El valor está concentrado en el 2 % del fuente, y ese 2 % está gateado

Las 39 funciones declaradas puras por `auditar_aislamiento.mjs` — `asignarLineas`, `verifDecision`, `prorratearOperacion`, `asignarGiros`, `recortarAsignacion` ...— suman 631 líneas: el 2 % del archivo. Ahí vive todo lo que un error convertiría en plata mal calculada.

Sobre ese 2 % hay 140 casos de suite. Es la proporción correcta y conviene decirla en voz alta: **el esfuerzo de prueba está donde está el riesgo de negocio**, no repartido por igual.

El otro 98 % es pantalla, catálogos y pegamento. Se verifica distinto —29 casos e2e y 151 gates de texto— y está bien que así sea.

Un número que lo confirma desde otro ángulo: la suite nombra 311 de los 954 símbolos de nivel módulo (33 %), pero la proporción se parte en dos mitades muy distintas — 306 de 829 símbolos que no son componentes (37 %) y 5 de 125 componentes (4 %). La suite no monta la UI por diseño; para eso está la capa e2e, que es de esta semana.

3. El riesgo real no es el tamaño: es el ESTADO

	PIPELINECOMERCIAL	DEALDRAWER
Líneas	3.023 (11,5 %)	2.862 (10,9 %)
<code>useState</code>	50	52
<code>useEffect</code> · <code>useRef</code> · <code>useMemo</code>	18 · 21 · 13	3 · 0 · 0
Anidamiento máximo	15 niveles	25 niveles
Funciones declaradas dentro	119	—
Comentario	23 %	17 %

Los dos juntos son el **22 % del fuente**. Pero el problema no es que sean largos: es que **concentran 102 de los 400 `useState` del archivo** y que uno de ellos llega a 25 niveles de anidamiento, contra una mediana de 1 y un p90 de 5 en el resto del archivo.

Consecuencias que ya se pagaron, medidas en esta misma semana:

- **La suite no puede montarlos.** Por eso existe `tests/e2e/`: las reglas de pantalla necesitaron un navegador con sesión iniciada porque el estado no se puede inyectar. Es una capa entera —16 archivos, 29 casos, 8 minutos de corrida— que existe por la forma de estos dos componentes.
- **Un cambio adentro no se puede revisar leyendo el diff.** Los siete defectos que los gates destaparon al cerrar la tabla de invariantes vivían todos ahí: el veto de retirar la última factura, el `setReevalPend` que faltaba, los dos `revertir` con identificadores fuera de alcance.

Lo bueno: **74 globales mutables, y 55 de ellas se escriben en dos sitios o menos**. La más escrita tiene seis sitios (`LOG_FH`). El acoplamiento por variable global, que sería el problema esperable en un archivo así, **no existe**: el estado está dentro de los componentes, no suelto.

4. Un defecto accionable: el paso 2 de la verificación no ve 11 declaraciones

Este es el único hallazgo de este informe que pide una corrección concreta.

El paso 2 de `CLAUDE.md` —el que busca símbolos duplicados de nivel módulo— es:

```
grep -oE "^(function|const|let|var) [A-Za-z0-9_]+" pipeline_comercial.jsx | awk '{print $2}' | sort | uniq -d
```

Ese patrón ve **944** declaraciones y **no ve 11**: diez `async function` (`sha256Hex`, `validarOtp`, `verificarAuditoria`, `vincularArchivoLog` ...) y el `export default function PipelineComercial`.

`auditar_muerto.mjs` sí las ve: su regex incluye `export default` y `async`, y el comentario que lo acompaña explica por qué, con el incidente que lo motivó —«un analizador que no ve una forma de declarar funciones no da un falso negativo: da un falso POSITIVO, que acá significa borrar código vivo».

Dos herramientas del mismo repo tienen definiciones distintas de «declaración», y la más débil es la que está en la ruta obligatoria de verificación.

El modo de falla es exactamente el que `code_style.md` documenta con dos incidentes: un `const x` que colisiona con un `async function x` es `SyntaxError: Identifier 'x' has already been declared`, `tsc` **no lo detecta** —está probado en este repo— y el paso 2 tampoco lo vería. Quedaría para el paso 5, cuando la suite no logra montar la app, a dos minutos de distancia y sin decir qué símbolo es.

La corrección es un carácter de regex, y el gate de contrato que fija la forma del fuente (`fuelle.test.mjs`) puede exigir que los dos patrones coincidan, con su sonda negativa.

5. Lo que NO es un problema (y conviene dejar escrito)

- **Duplicación:** 0 bloques repetidos de 15 líneas; 0,4 % del archivo con ventana de 8. En un fuente de este tamaño es un resultado inusual.
- **Unidades:** `auditar_unidades.mjs` da **0 candidatos**. La regla del peso entero, que costó un incidente de \$75 millones de error acumulado, se sostiene sola.
- **Código muerto:** 6 hallazgos «revisar a mano» y 7 `useState` sin uso, todos en la línea base y ninguno nuevo. La poda del 11-09 sigue firme.
- **Globales:** 74, casi todas con dos escrituras o menos (§3).
- **Comentarios:** 19 % del archivo. Es memoria, no ruido: buena parte de las reglas del vault salieron de ahí.

6. Qué haría, en orden

1. **Cerrar el agujero del paso 2** (§4). Es un regex y su gate. Media hora, y elimina un modo de falla que ya ocurrió dos veces por otras vías.
2. **Nada más sobre el código.** Ni partir `DealDrawer`, ni bajar los `useState`, ni tocar el anidamiento. Un refactor de los dos componentes grandes es el cambio más caro y riesgoso que este repo admite —toca el 22 % del fuente, lo revisa un diff ilegible, y lo único que lo sostendría son 29 casos e2e de ocho minutos—. **El costo no se justifica con la evidencia que hay:** no hay un incidente causado por el tamaño de esos componentes, sólo por reglas que nadie había cableado, y esas ya están cableadas.

Si algún día hay que tocarlos, la condición previa está clara y es medible: que la capa e2e cubra el detalle lo suficiente como para que un refactor se pueda probar sin abrir Chrome a mano.

7. Lo que este análisis NO cubre

- **Rendimiento en el navegador:** nadie lo ha medido. Un archivo de 40,7 MB con Babel transpilando en el cliente tiene un costo de arranque que este informe no tocó.
- **Accesibilidad:** 12 % de las líneas llevan `style` inline y la navegación por teclado no está probada fuera del Command-K.
- **Los 20 desfases regla ↔ código** que los gates midieron: están en `vault/sesiones/2026-09-17_cerrar_invariantes.md` y son decisión del usuario, no hallazgos de forma.

8. Cómo reproducir cada número

```

wc -l pipeline_comercial.jsx # 26.233
grep -cE "^(function|const|let|var) [A-Za-z0-9_]+" pipeline_comercial.jsx # 944 (el paso 2)
grep -cE "^(export|class|async function) " pipeline_comercial.jsx # 11 (las que no ve)
grep -c "useState(" pipeline_comercial.jsx # 400
grep -cE "^let " pipeline_comercial.jsx # 74 globales mutables
node auditar_muerto.mjs # 6 + 7, la línea base
node auditar_unidades.mjs # 0 candidatos
node auditar_aislamiento.mjs # las 39 puras

```

Los censos por cuerpo real (tamaños, anidamiento, duplicación, cobertura de la suite) se hicieron con tres scripts de medición; su lógica está descrita en cada sección y se rehacen en minutos. No se versionan: miden, no gatean.